CENTRO UNIVERSITÁRIO IESB
Centro Universitário Instituto de Educação Superior de Brasília
Coordenação de Engenharia de Computação

ANÁLISE COMPARATIVA DO DESEMPENHO DE COMPILADORES EM ALGORITMOS DE REGRESSÃO LINEAR

Autor: Dannyel Fontenele Ribeiro | dannyelfr@yahoo.com.br
Orientador: Esp. Roger Rocha Ferreira | uni.pro.frogger@gmail.com

OBJETIVO

Medir até onde compiladores reduzem a diferença entre equações normais (NE) ingênuas e a decomposição QR de Householder do OpenBLAS em regressão linear por mínimos quadrados ordinários na plataforma AMD Zen 5, sendo o segundo método a referência. Além disso, observar a evolução dos *speedups* entre níveis de otimização e comparar a estabilidade numérica dos dois métodos.

INTRODUÇÃO

Mínimos quadrados podem ser resolvidos por equações normais ($X^TX = X^Ty$), que são mais baratas, ou por decomposição QR de Householder, que é mais estável, pois X^TX eleva o número de condição (sensibilidade a erros) ao quadrado. Compiladores conseguem otimizar código próprio, enquanto o OpenBLAS provê rotinas otimizadas que aproveitam melhor o *hardware*. Então, deve-se determinar até que ponto os compiladores fecham essa diferença de tempo e a que custo de precisão, caso tenha.

METODOLOGIA

Os experimentos foram realizados no Ubuntu 26.04 no Subsistema Windows para Linux 2 no Windows 11 25H2 com um processador AMD Ryzen 9 9950X3D usando os compiladores GCC 15 e Clang 21 e a biblioteca OpenBLAS 0.3.32 em modo *single thread*.

Os dados sintéticos modelaram um problema de regressão linear, de mil a um milhão de linhas (pontos de dados, ou n) e de 11 a 101 colunas (variáveis, ou p), e com número de condição (κ) de 12 a 95 (conjunto bem condicionado) até 16 mil (mal condicionado), armazenados em arquivos binários em ordem de linha. Houve também um teste de estabilidade com 10 mil linhas e κ variando de 10 a 100 milhões.

NE foi implementado de forma ingênua em C com fatoração de Cholesky e compilado 6 vezes (GCC e Clang, otimizações O0, O1, O3+native), enquanto QR usou a interface LAPACKE da OpenBLAS, cuja medição de tempo incluía a transposição interna para ordem de coluna. Foram feitos 3 aquecimentos e 20 execuções cronometradas, tomando a mediana destas.

RESULTADOS E DISCUSSÕES

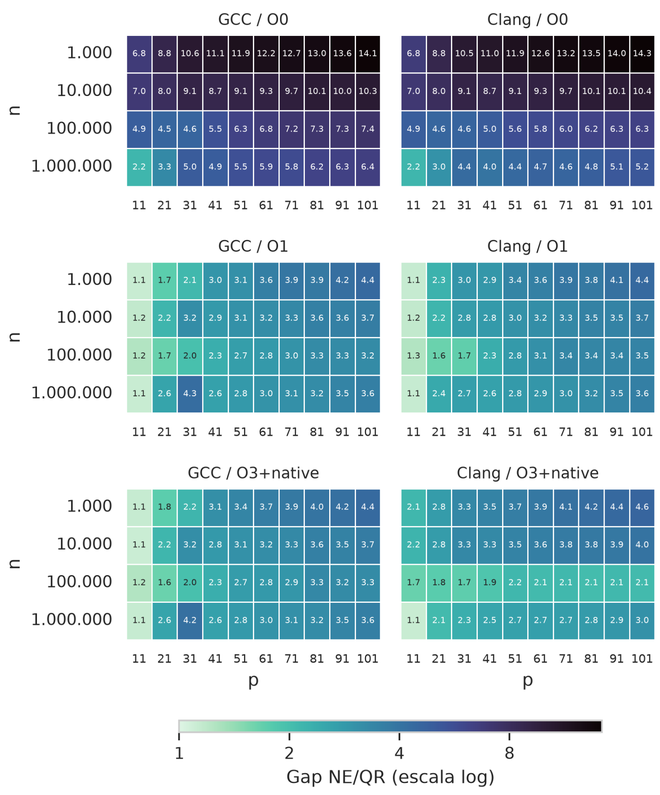


Figura 1 – Razões de Tempo (NE/QR)

Em O0, NE é 4,5 a 12,2 vezes mais lento que QR, 2,8 a 3,5 vezes em O1 e 2,1 a 3,8 vezes em O3+native. Essa lacuna depende mais da quantidade de variáveis do que da quantidade de pontos de dados: quando n = um milhão, NE é entre 1,1 e 3,6 vezes mais lento que QR.

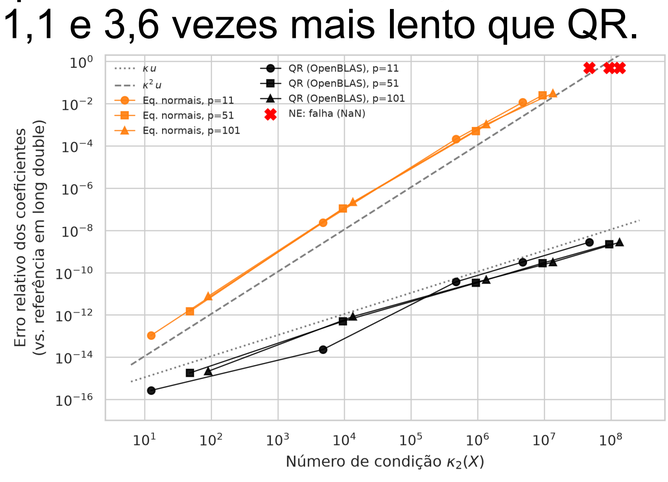


Figura 2 – Erro vs. κ

O erro de NE segue κ^2, com até 3% de erro antes de o Cholesky falhar quando κ fica acima de cerca de 50 milhões, enquanto QR segue κ e não passa de 3e-9. Os resíduos dos dois métodos foram praticamente os mesmos.

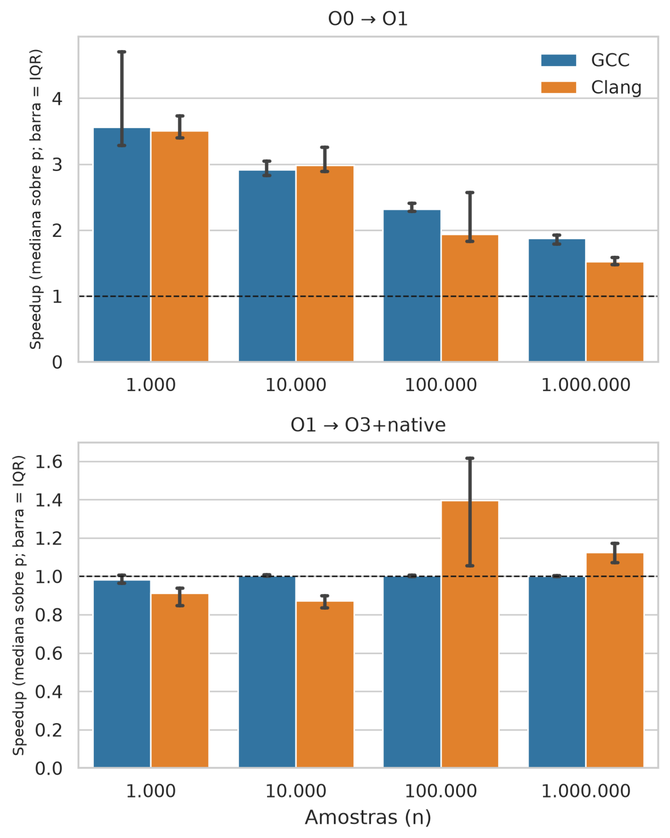


Figura 3 – *Speedups* entre otimizações

O ganho de velocidade vem quase todo da transição de O0 para O1, de 3,5 vezes quando n = mil até 1,5 a 1,9 vez quando n = um milhão, enquanto O3+native alterou nada no GCC e foi inconsistente no Clang. Nenhum compilador vetorizou a formação de X^TX, a parte mais cara, que custa $O(n \cdot p^2)$.

CONCLUSÃO

A otimização de compiladores reduz, mas não fecha a diferença entre os métodos, continuando de 1,1 a 3,6 vezes mais lento que a referência. Os ganhos se concentram na primeira transição de otimização, e a segunda foi inconsistente. Quanto à estabilidade, NE acompanhou κ^2 e falhou com κ alto, enquanto QR acompanhou o número. Logo, a vantagem teórica de NE depende da qualidade da implementação.

REFERÊNCIAS

Artigo completo, referências e código: https://pastebin.com/F8y8aEJ3